

- [1. lib/api.ts et middleware.ts — Le middleware d'interception](#)
- [2. Leaflet.js — N'existe pas dans notre projet](#)
- [3. Injection des en-têtes contextuels BCaaS](#)
- [4. Endpoints et fonctions métier](#)
- [5. Next.js 15 — Pourquoi ce choix ?](#)
- [6. Stockage des données \(pas de BD\)](#)
- [7. Ce qui permet les simulations avec des mock data](#)
- [8. Rendu hybride SSR/SSG/ISR](#)
- [9. Gestion native des imports dynamiques](#)

Je vais répondre point par point en m'appuyant sur le rapport ([main.tex](#)) et le code réel du projet.

1. [lib/api.ts](#) et [middleware.ts](#) — Le middleware d'interception

Ces fichiers n'existent pas dans notre projet actuel.

- [lib/api.ts](#) est décrit dans le rapport comme la "couche d'abstraction centrale entre les composants React et le backend BCaaS". Il devait injecter les en-têtes contextuels ([X-Tenant-Id](#), [X-Trace-Id](#), [role_scope](#)) dans chaque requête. **Dans notre projet**, l'équivalent est [lib/demoStorage.ts](#) qui gère directement le localStorage.
- [middleware.ts](#) est un fichier spécial Next.js placé à la racine de [src/](#). Il intercepte **chaque requête HTTP entrante** avant qu'elle n'atteigne une page. Dans le rapport, il devait :
 - Vérifier la présence d'un token JWT dans les cookies
 - Rediriger vers [/connexion](#) si l'utilisateur n'est pas connecté
 - Injecter l'en-tête [X-Tenant-Id: painting](#)

Dans notre projet, l'équivalent est [DemoStoreContext.tsx](#) qui vérifie la session côté client, et les composants [PainterGate/AdminGate](#) qui protègent les routes.

2. Leaflet.js — N'existe pas dans notre projet

Leaflet est une bibliothèque JavaScript open-source pour afficher des **cartes géographiques interactives**. Le rapport en parle comme une fonctionnalité prévue pour géolocaliser les peintres sur une carte, mais elle n'a pas été implémentée. Elle aurait dû être importée dynamiquement pour éviter les erreurs SSR (rendu serveur).

3. Injection des en-têtes contextuels BCaaS

C'est le mécanisme qui **encapsule chaque requête** avec des métadonnées standardisées, comme le ferait un protocole réseau. Dans la pile BCaaS à 5 couches :

En-tête	Exemple	Rôle
X-Tenant-Id	painting	Identifie notre instance métier
actor_id	peintre-007	Qui émet la requête
role_scope	PAINTER	Droits de l'utilisateur
trace_id	UUID v4	Traçabilité bout en bout
locale	fr-CM	Langue/région

Dans notre projet, c'est simulé via le `DemoStoreContext` : la session contient `userId`, `role`, `email`, etc. — ce sont nos "en-têtes contextuels" version simplifiée.

4. Endpoints et fonctions métier

Dans le rapport, les fonctions prévues dans `lib/api.ts` étaient :

Fonction	Endpoint prévu	Équivalent dans notre code
<code>getFiches()</code>	GET <code>/api/v1/ressources</code>	<code>data/siteContent.ts</code> → <code>fiches[]</code>
<code>getFicheById()</code>	GET <code>/api/v1/ressources/{id}</code>	<code>useDemoStore().getFicheBySlug(slug)</code>
<code>creerFiche()</code>	POST <code>/api/v1/ressources</code>	<code>saveFicheDraft()</code> / <code>submitFicheForReview()</code>
<code>modifierFiche()</code>	PUT <code>/api/v1/ressources/{id}</code>	Pareil (upsert dans le store)
<code>rechercher()</code>	GET <code>/api/v1/search?q=</code>	Filtrage client dans <code>FichesPage.tsx</code> avec <code>useMemo</code>
<code>getRoadmaps()</code>	GET <code>/api/v1/roadmaps</code>	<code>roadmaps[]</code> depuis <code>siteContent/roadmaps.ts</code>
<code>login()</code>	POST <code>/api/v1/auth/login</code>	<code>useDemoStore().login(email, password)</code>
<code>register()</code>	POST <code>/api/v1/auth/register</code>	<code>useDemoStore().register(name, email, password)</code>
<code>getActeurs()</code>	GET <code>/api/v1/actors</code>	<code>getAllRegisteredUsers()</code> dans le store
<code>validerCompte()</code>	PUT <code>/api/v1/actors/{id}/valider</code>	Non implémenté (admin manuel)

5. Next.js 15 — Pourquoi ce choix ?

Next.js est un framework React qui ajoute du rendu serveur (SSR), du routage par système de fichiers, et des optimisations automatiques.

Pourquoi Next.js 15 plutôt que 14 ?

Fonctionnalité	Next.js 14	Next.js 15
@next/codemod upgrade	Manuelle	Automatique via codemod
React 19 support	Non (React 18)	Oui (React 19 canary)
Formulaires serveur	Non	actions.ts natifs
Cache HTTP avancé	Basique	Amélioré
create-next-app	Pages Router	App Router par défaut
Hydratation partielle	Non	Oui (Partial Prerendering)

Notre projet utilise Next.js 15 côté App Router, ce qui nous donne :

- Routes automatiques via app/fiches/[slug]/page.tsx
- Composants serveur (Server Components) par défaut
- SEO optimisé (métadonnées dans generateMetadata())

6. Stockage des données (pas de BD)

On stocke tout dans localStorage via lib/demoStorage.ts :

Donnée	Clé localStorage	Format
Session utilisateur	artplastique-demo-session	JSON (DemoSession)
Fiches + roadmaps	artplastique-demo-data	JSON (DemoData)
Peintres inscrits	artplastique-registered-users	JSON (DemoUser[])

Le localStorage survit au rechargement de page et à la fermeture du navigateur. Pour réinitialiser, il suffit d'appeler resetDemoData().

7. Ce qui permet les simulations avec des mock data

C'est React Context + localStorage qui permet la simulation :

```
DemoStoreProvider (context/DemoStoreContext.tsx)
├─ Au démarrage : seedFictionalPainters() + loadSession() + loadDemoData()
├─ Fournit : session, fiches, roadmaps, login(), register(), etc.
```

```
└─ Persiste via : saveSession() + saveDemoData() + upsertRegisteredUser()
    └─ localStorage (navigateur)
```

Tous les composants UI appellent `useDemoStore()` pour lire/écrire des données, exactement comme ils appelleraient une API REST. Le remplacement par un vrai backend se ferait en modifiant uniquement `DemoStoreContext.tsx` — les composants UI ne changent pas.

8. Rendu hybride SSR/SSG/ISR

C'est la capacité de Next.js à choisir **par page** comment générer le HTML :

Stratégie	Signification	Dans notre projet
SSR (Server-Side Rendering)	HTML généré à chaque requête côté serveur	Dashboard peintre (données dynamiques)
SSG (Static Site Generation)	HTML généré une fois au build	Pages publiques (fiches, fiches/[slug])
ISR (Incremental Static Regeneration)	SSG + re-génération périodique	Non utilisé ici

Dans notre projet : comme on est en mode 100% client (pas de serveur Node.js), tout est rendu côté client (`'use client'`). Le SSR/SSG/ISR nécessiterait un déploiement sur Vercel ou un serveur Node.

9. Gestion native des imports dynamiques

C'est la fonction `dynamic()` de Next.js qui permet de **charger un composant uniquement côté client** (pas pendant le rendu serveur) :

```
import dynamic from 'next/dynamic'

const MapView = dynamic(() => import('@components/MapView'), {
  ssr: false, // ← évite l'erreur "window is not defined" pour Leaflet
  loading: () => <p>Chargement de la carte...</p>
})
```

Utilité dans notre projet : si on intégrait Leaflet.js (bibliothèque de cartes), `dynamic()` empêcherait Next.js de tenter de charger Leaflet côté serveur (où `window` et `document` n'existent pas), tout en affichant un squelette de chargement pendant le téléchargement.

Équivalent actuel dans notre projet : on utilise simplement `'use client'` en haut des composants qui ont besoin du navigateur. Pas besoin de `dynamic()` car tout notre code est en mode client.

Besoin que je développe un point en particulier ? Passe-moi en **ACT MODE** si tu veux que je crée `middleware.ts` ou `lib/api.ts` pour de vrai.